

Sistema de adquisición EEG de 16 canales con OpenBCI

Sistema desarrollado para la **adquisición y almacenamiento de señales electroencefalográficas (EEG)** utilizando un dispositivo **OpenBCI Cyton con expansión Daisy**, conectado a una **Raspberry Pi** mediante comunicación serial y programado en Python.

El proyecto permite adquirir simultáneamente información de **16 canales EEG**, interpretar los paquetes de datos enviados por el dispositivo, convertir las muestras digitales a valores correspondientes de amplitud y generar una representación gráfica de las señales adquiridas.

Descripción

El sistema utiliza un:

- **OpenBCI Cyton** como dispositivo principal de adquisición.
- **OpenBCI Daisy** para ampliar el número de canales.
- **Raspberry Pi** como plataforma de procesamiento.
- **Python** para la comunicación, adquisición y procesamiento de los datos.

La comunicación entre la Raspberry Pi y el OpenBCI se realiza mediante un puerto serial utilizando una velocidad de:

115200 baudios

El programa configura el dispositivo, inicia la transmisión de datos y procesa los paquetes recibidos para obtener las señales correspondientes a los 16 canales.

Objetivo

El objetivo principal del proyecto es desarrollar un sistema capaz de:

1. Establecer comunicación serial con el OpenBCI Cyton.
2. Inicializar y configurar el dispositivo.
3. Iniciar la transmisión de datos EEG.
4. Recibir los paquetes enviados por el OpenBCI.
5. Extraer los datos correspondientes a los 16 canales.
6. Interpretar las muestras de 24 bits.
7. Almacenar los datos adquiridos en un archivo CSV.
8. Convertir las muestras a una escala de microvoltios.

9. Generar una gráfica de las señales EEG de los 16 canales.

Hardware

OpenBCI Cyton

El **OpenBCI Cyton** es la tarjeta utilizada para adquirir las señales bioeléctricas provenientes de los electrodos.

En este proyecto se utiliza como dispositivo principal de adquisición EEG.

OpenBCI Daisy

La expansión **Daisy** permite ampliar el número de canales disponibles, haciendo posible trabajar con un total de 16 canales.

La información de los canales se recibe mediante paquetes de datos que posteriormente son procesados por el programa.

Raspberry Pi

La Raspberry Pi funciona como plataforma de procesamiento.

Sus principales funciones dentro del proyecto son:

- Establecer la comunicación serial.
- Enviar comandos al OpenBCI.
- Recibir los paquetes de datos.
- Procesar las muestras.
- Almacenar los datos.
- Generar las gráficas.

Software

El proyecto está desarrollado en **Python** y utiliza las siguientes bibliotecas:

Biblioteca	Función
NumPy	Manejo de arreglos y datos numéricos
PySerial	Comunicación serial con OpenBCI

Biblioteca	Función
time	Control de tiempos durante la inicialización
Pandas	Organización y almacenamiento de datos
Matplotlib	Visualización de las señales EEG

Dependencias

Las dependencias utilizadas por el proyecto son:

```
numpy
pyserial
pandas
matplotlib
```

Pueden instalarse mediante:

```
pip install numpy pyserial pandas matplotlib
```

Comunicación con OpenBCI

El programa establece la conexión utilizando:

```
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
```

Esto indica que el dispositivo OpenBCI está disponible mediante:

```
/dev/ttyUSB0
```

y que la comunicación serial utiliza:

```
115200 baudios
```

Después de establecer la conexión, el programa espera dos segundos para permitir la inicialización de la comunicación:

```
time.sleep(2)
```

Inicialización del dispositivo

Una vez establecida la comunicación, el programa envía un comando de reinicio:

```
ser.write(b'v')
```

Posteriormente se espera un segundo:

```
time.sleep(1)
```

Después se envía:

```
ser.write(b'd')
```

para utilizar la configuración predeterminada de los canales.

Inicio de adquisición

Antes de comenzar la adquisición se limpia el búfer de entrada:

```
ser.reset_input_buffer()
```

Esto permite descartar datos que pudieran haber quedado almacenados anteriormente en el puerto serial.

Posteriormente se envía:

```
ser.write(b'b')
```

para iniciar el streaming de datos.

Adquisición de 16 canales

El programa está configurado para adquirir:

```
N_points = 1000
```

muestras.

Para almacenar la información se crea una matriz:

```
data_EEG = np.zeros((16, N_points))
```

La estructura de esta matriz es:

	Muestras			
	0	1	2	... 999
Canal 1				
Canal 2				
Canal 3				
...				
Canal 16				

Por lo tanto:

```
16 filas      → 16 canales  
1000 columnas → 1000 muestras
```

El acceso:

```
data_EEG[ch, i]
```

representa el valor del canal `ch` durante la muestra `i`.

Lectura de paquetes

El programa lee dos paquetes consecutivos:

```
packet1 = ser.read(33)
packet2 = ser.read(33)
```

Cada paquete contiene 33 bytes.

Antes de procesarlos se verifica que ambos paquetes tengan el tamaño esperado:

```
if len(packet1) == 33 and len(packet2) == 33:
```

También se verifica el byte inicial del paquete:

```
if packet1[0] == 0xA0 and packet2[0] == 0xA0:
```

El valor:

```
0xA0
```

se utiliza para identificar el inicio de un paquete de datos válido.

Obtención de los canales 1-8

Los primeros ocho canales se extraen del primer paquete:

```
for ch in range(8):
    idx = 2 + ch*3
    data_EEG[ch, i] = read_24bit_signed(
        packet1[idx],
        packet1[idx+1],
        packet1[idx+2]
    )
```

El ciclo:

```
for ch in range(8):
```

genera los valores:

```
0, 1, 2, 3, 4, 5, 6, 7
```

Cada canal utiliza tres bytes consecutivos.

La posición inicial se calcula mediante:

```
idx = 2 + ch*3
```

Por lo tanto, las posiciones utilizadas son:

Canal	Bytes
Canal 1	2, 3, 4
Canal 2	5, 6, 7
Canal 3	8, 9, 10
Canal 4	11, 12, 13
Canal 5	14, 15, 16
Canal 6	17, 18, 19
Canal 7	20, 21, 22
Canal 8	23, 24, 25

Obtención de los canales 9–16

Los canales restantes se obtienen del segundo paquete:

```
for ch in range(8):  
    idx = 2 + ch*3  
    data_EEG[ch+8, i] = read_24bit_signed(  
        packet2[idx],  
        packet2[idx+1],  
        packet2[idx+2]  
    )
```

El uso de:

```
ch + 8
```

permite almacenar los datos en las posiciones correspondientes a los canales 9–16.

Por ejemplo:

```
ch = 0 → canal 9  
ch = 1 → canal 10  
ch = 2 → canal 11  
...  
ch = 7 → canal 16
```

De esta manera, los dos paquetes se integran dentro de una única matriz de 16 canales.

Conversión de datos de 24 bits

Cada muestra de canal está representada mediante tres bytes, es decir:

```
3 bytes × 8 bits = 24 bits
```

Para interpretar estos tres bytes se utiliza la función:

```
def read_24bit_signed(b1, b2, b3):  
    value = (b1 << 16) | (b2 << 8) | b3  
  
    if value & 0x800000:  
        value -= 1 << 24  
  
    return value
```

Primero se combinan los tres bytes:

```
value = (b1 << 16) | (b2 << 8) | b3
```

El desplazamiento de bits permite colocar cada byte en su posición correspondiente dentro del valor de 24 bits.

Posteriormente se verifica el bit de signo:

```
if value & 0x800000:
```

Si dicho bit está activo, el valor se interpreta como negativo mediante complemento a dos:

```
value -= 1 << 24
```

Finalmente, la función devuelve el valor entero correspondiente a la muestra.

Almacenamiento de datos

Una vez finalizada la adquisición se detiene el streaming:

```
ser.write(b's')
```

y se cierra la comunicación:

```
ser.close()
```

Después se muestra el tamaño de los datos:

```
print("Datos obtenidos:", data_EEG.shape)
```

La matriz resultante tiene:

```
(16, 1000)
```

lo que corresponde a:

```
16 canales × 1000 muestras
```

Generación del archivo CSV

Los datos se convierten en un DataFrame de Pandas:

```
df = pd.DataFrame(  
    data_EEG.T,  
    columns=[f"Canal_{i+1}" for i in range(16)]  
)
```

Se utiliza:

```
data_EEG.T
```

para transponer la matriz.

Originalmente:

```
16 × 1000
```

Después de la transposición:

```
1000 × 16
```

Esto permite que cada fila represente una muestra y cada columna represente un canal.

El archivo se guarda mediante:

```
df.to_csv("EEG_16_canales.csv", index=False)
```

El resultado es:

```
EEG_16_canales.csv
```

con una estructura similar a:

```
Canal_1, Canal_2, Canal_3, ..., Canal_16  
muestra, muestra, muestra, ..., muestra  
muestra, muestra, muestra, ..., muestra  
...
```

Conversión a microvoltios

Después de almacenar los valores originales, el programa realiza una conversión opcional a microvoltios:

```
scale = 4.5 / (24 * (2**23 - 1)) * 1e6  
data_EEG_uV = data_EEG * scale
```

El factor `scale` permite convertir los valores digitales obtenidos del ADC a una escala expresada en:

μV

Los valores convertidos se almacenan en:

`data_EEG_uV`

Esta conversión se utiliza posteriormente para representar gráficamente las señales.

Generación de la gráfica

Para generar el eje temporal se utiliza:

```
t = np.arange(N_points) / fs
```

donde:

```
N_points = 1000  
fs = 250
```

El eje temporal se genera a partir del número de muestras y la frecuencia de muestreo utilizada para representar los datos.

Para evitar que las 16 señales aparezcan superpuestas, se aplica un desplazamiento vertical:

```
offset = 100
```

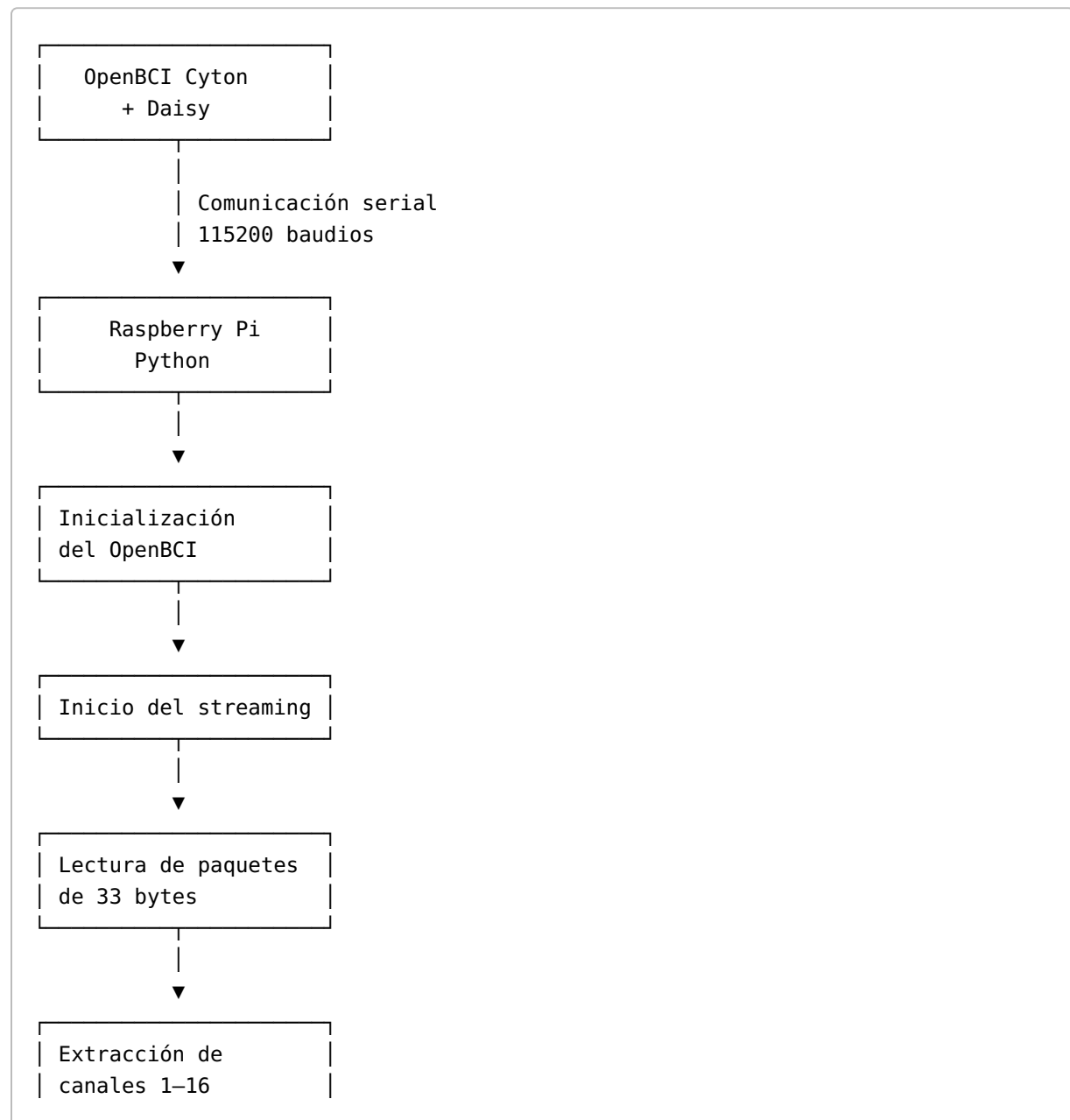
Posteriormente cada canal se grafica mediante:

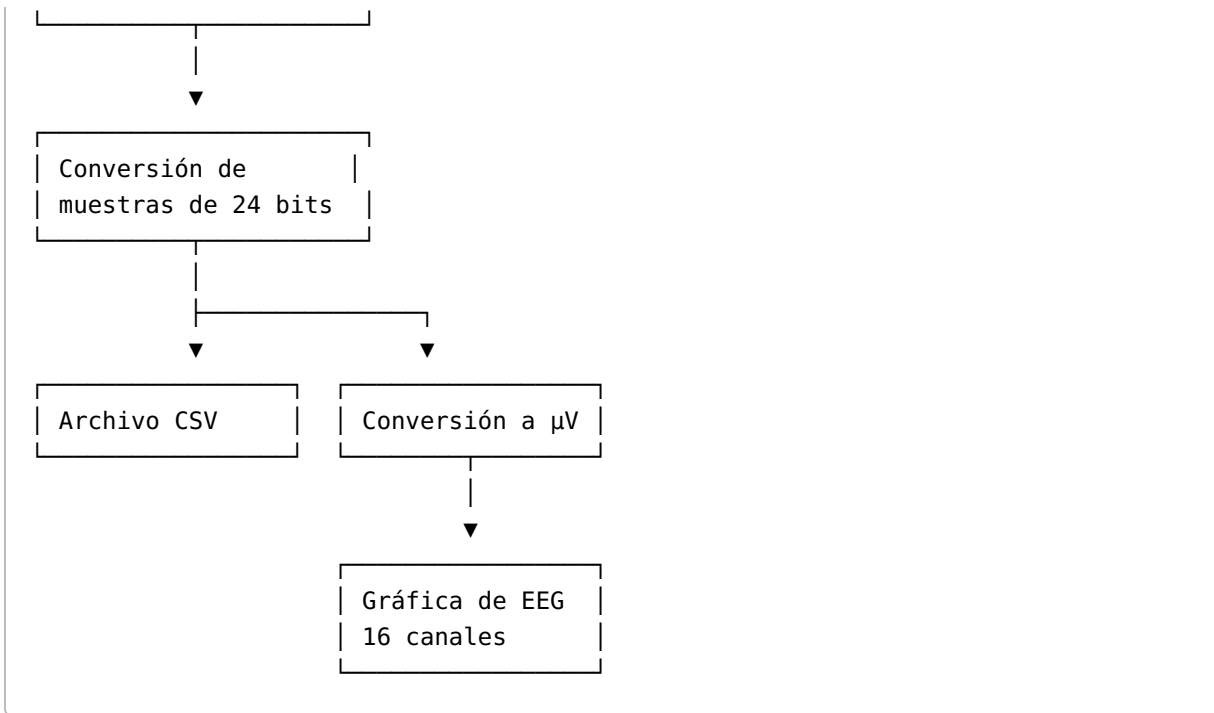
```
for ch in range(16):  
    plt.plot(t, data_EEG_uV[ch] + ch*offset)
```

Cada canal conserva su señal original, pero se desplaza verticalmente para facilitar su visualización.

Flujo general del sistema

El funcionamiento completo puede resumirse de la siguiente manera:





Ejecución

Desde la Raspberry Pi, ejecutar:

```
python3 adquisicion_eeg_16ch.py
```

El programa realizará automáticamente:

1. Apertura del puerto serial.
2. Inicialización del OpenBCI.
3. Configuración de los canales.
4. Inicio del streaming.
5. Adquisición de 1000 muestras.
6. Procesamiento de los 16 canales.
7. Detención del streaming.
8. Generación del archivo CSV.
9. Conversión de los datos a microvoltios.
10. Generación de la gráfica.

Resultados

El sistema genera principalmente dos resultados:

Archivo de datos

```
EEG_16_canales.csv
```

Contiene las muestras adquiridas correspondientes a los 16 canales.

Visualización

El programa genera una gráfica con las señales correspondientes a:

```
Canal 1  
Canal 2  
Canal 3  
...  
Canal 16
```

Las señales se muestran con un desplazamiento vertical para facilitar la comparación visual.

Consideraciones

Este proyecto está orientado principalmente a la **adquisición, almacenamiento y visualización de señales EEG**.

El código actual no implementa todavía etapas avanzadas de procesamiento como:

- Filtrado digital EEG.
- Eliminación de artefactos.
- Análisis de frecuencia.
- Transformada de Fourier.
- Cálculo de potencia espectral.
- Detección automática de ondas cerebrales.
- Clasificación de estados cerebrales.

Estas funcionalidades pueden incorporarse posteriormente como etapas adicionales de procesamiento.

Trabajo futuro

Algunas posibles extensiones del proyecto son:

- Implementar filtros pasa-banda para señales EEG.
 - Implementar filtros notch para interferencia de la red eléctrica.
 - Detectar y eliminar artefactos.
 - Analizar las bandas Delta, Theta, Alpha, Beta y Gamma.
 - Implementar análisis mediante FFT.
 - Generar espectrogramas.
 - Crear una interfaz gráfica para monitoreo en tiempo real.
 - Implementar adquisición continua.
 - Integrar el sistema con bases de datos.
 - Desarrollar algoritmos de clasificación de señales EEG.
-

Tecnologías utilizadas

```
Python
NumPy
PySerial
Pandas
Matplotlib
Raspberry Pi
OpenBCI Cyton
OpenBCI Daisy
EEG
Comunicación Serial
```

Autor

Proyecto desarrollado como parte del **servicio social**, enfocado en la adquisición y procesamiento de señales electroencefalográficas mediante hardware OpenBCI y Python.